

Unity Motion Matching (MxM) Locomotion and Parkour Documentation

None

Table of contents

1. Overview	3
1.1 Included Features:	3
1.2 What this asset is:	3
1.3 What this asset is not :	3
1.4 Highly-Recommended Optional Integrations Included:	3
1.5 Major Components	4
1.6 Folder Structure	4
2. Installation	5
2.1 Supported Unity/Pipeline Version	5
2.2 Definitions and Notes	5
2.3 Creating Project and Installing Dependencies	5
2.4 Installation steps - 1 - Project Creation and Dependency Packages	6
2.5 Installation steps - 2 - Project Settings	6
2.6 Installation steps - 3	6
2.7 Installation - Optional Animation Sets	7
3. Configuration - Characters	10
3.1 Configuration Wizard	10
3.2 Setting up an input-driven character (Player)	12
3.3 Setting up a Non-Player character (AI/NPC)	13
3.4 Tuning Character Behavior for Players and NPC's	13
3.5 Setting up your Character and Environment for Final IK grounder	20
3.6 Using any of the demo scenes that include Final IK	26
4. Configuration - Final IK Grounder	29
4.1 Setting up your Character and Environment for Final IK grounder	29
4.2 Using any of the demo scenes that include Final IK	34
5. Demo Scenes and Examples	37
5.1 Completed Examples	38
5.2 Completed Tutorials	38
5.3 Scripts from Youtube Tutorials	38

1. Overview

This controller allows you to achieve the AAA-level character locomotion quality of motion matching without the learning curve. All character configuration and tuning is handled by our controller and meant to be easily integrated into your existing mechanics (with a small amount of scripting knowledge).



This asset requires Motion Matching for Unity (MxM) and Kubold's Movement Animset Pro and can't work without them.

It combines (and requires) the fantastic Motion Matching for Unity (MxM) asset and Kubold's Movement Animset Pro with custom animations and code to provide realistic AAA-grade crouch, walk, run, sprint, procedural jumping, falling, and landing for your games and projects without having to learn how to configure motion matching yourself.

1.1 Included Features:

- Idles, and all-direction crouch, walk, run, and sprint
- Procedural Jump, midair, and landing behavior
- Automatic landing variants: normal, hard landing with clear forward path (front roll), hard landing with obstructed path (stationary landing to idle)
- Configuration wizard to create new fully-configured characters from your models or modify existing characters in your scenes
- Easy blend in/out of motion-matching mode to integrate with your other character behaviors, animations, and controllers.

1.2 What this asset is:

- A well-tuned and configured motion-matching locomotion setup for humanoid characters and players to crouch, walk, run, and sprint like the demo video, which can be integrated via scripting with other controllers, functionality, and toolkits to provide a full game/project experience
- Scripts and behaviors for realistic procedural jumping, falling, and landing
- A configuration wizard for configuring this controller with your humanoid character models
- A mechanism to leverage the amazing power and movement precision of motion-matching without having to deeply learn and configure the motion matching asset. Extension and modification of the motion-matching or other behaviors of this controller requires coding and knowledge of the assets involved.

1.3 What this asset is not:

- A zero-lift all-in-one player and character controller: It handles locomotion and that's all for now
- A no-code integration solution (if you're looking for low- or no-code, it's probably best to wait for our Game Creator 2 integration (coming soon!) or request an integration with your preferred game toolkit)

ALWAYS MAKE A BACKUP OF YOUR PROJECTS AND SCENES BEFORE IMPORTING/USING THIS ASSET!

1.4 Highly-Recommended Optional Integrations Included:

- Strider for smooth transitions from idle to walk, walk to run, and run to sprint.
- Final IK (For Grounder, but Full Body Biped IK rig is configured automatically for your character as part of our configuration wizard) for proper foot placement on uneven terrain and ground surfaces (like stairs).

**Note**

Requests for integrations with existing controllers, game toolkits, and other assets should be made in the integration request channel of our discord (link below) and, if enough users want the feature, they'll be included in an update or split out as a separate integration asset based on difficulty/complexity of the integration and amount of predicted support required.

There is no affiliation between this package/publisher and Kubold, but this package was independently built to utilize Movement Animset Pro

This asset is in active development, with more features to be added periodically based both on planned additions and user feedback (see Discord link below to give feedback)!

For support and requests, use our Discord or email threepeatgames@gmail.com.

1.5 Major Components

MMLC is organized into two major components:

- NGCharacterBase -> NGPlayerController and NGNonPlayerController: Handles characters' main business logic for jumping, fall detection, landing, and other abilities (coming soon)
- NGCharacterControllerWrapper: Handles MxM and MMLC's interaction with the Unity CharacterController to move the character

The behavior of MMLC is driven by configuration of these components, and through the use of configuration Scripted Objects:

- NGJumpSettings: Contains all configuration for jumping, falling, and landing

1.6 Folder Structure

MMLC imports into **Assets\Plugins\Threepeat\MMCLocomotion** with the following structure, but feel free to move them wherever makes sense for your project:

- Plugins/Threepeat/MMCLocomotion
- **Animations**: contains all source animations for jumping and landing. All other animations come from the (not included) Movement Animset Pro.
- **Configs**: contains jump settings scriptable objects and InputManager presets.
- **models**: Any models, textures, materials, and other assets necessary for the asset
- **Demos**: Example Scenes, for testing MMLC install
- **Editor**: Custom Inspectors and Editor Windows
- **MotionMatching**: All MxM-related configuration files for MMLC
- **Integrations**: Contains importable unity packages for Final IK and Strider integrations (more to come!)
- **Scripts**: contains all source code

2. Installation

2.1 Supported Unity/Pipeline Version

This asset works in 2019+, but 2021 LTS or later is recommended.

2.2 Definitions and Notes

- **MMLC** - Motion-Matching Locomotion Controller (This asset)
- **MxM** - Motion-Matching for Unity (Animation Uprising) asset
- **MAP** - Movement Animset Pro (Kubold) asset

2.3 Creating Project and Installing Dependencies

The packages below are required and must be installed prior to the Motion-Matching Locomotion Controller (MMLC).



Note

Everything in Installations steps 1 and 2 are for dependency packages and not directly related to the MMLC asset itself. If you encounter any issues during the installation of these assets, please check with their respective forums, manuals, and communities for fastest assistance.

2.4 Installation steps - 1 - Project Creation and Dependency Packages



These instructions are for Unity 2021 LTS. If any of these steps don't work exactly as expected in your Unity version, google likely has you covered, but please reach out on discord if you're stuck.

If you prefer a video walkthrough of these steps, that can be found [here](#):

1. Create Unity Project with your desired pipeline (Skip this step if you already have a project)
2. (Optional, for speed): Edit -> Project Settings -> Editor -> Parallel Import: set to true
3. Make sure Burst is installed in Package Manager -> Packages: Unity Registry. Install if not already installed.
4. Make sure Mathematics is installed in Package Manager -> Packages: Unity Registry. Install if not already installed.
5. Install Collections in Package Manager -> Packages: Unity Registry
6. Install InputSystem in Package Manager -> Packages: Unity Registry
 - a. If you're not prompted to use the new input system (if you are, click YES), go to Project Settings -> Other Settings -> Configuration and set Active Input Handling to "Both". Another way to find this settings is to type **Active Input** into the search box in the upper right of the Project Settings window.
7. Use Package Manager -> Packages: Unity Registry -> + to add packages by name:
 - a. com.unity.playablegraph-visualizer
 - b. com.unity.jobs (if not present and installable from Unity Registry list)
8. Install Cinemachine from Package Manager -> Packages: Unity Registry
9. Install Motion Matching for Unity
10. (Optional, but highly recommended) Install Strider (Highly recommend clicking **Skip** on the dependencies popup, since we've already installed them and there's one (package-manager-ui) that is obsolete and will break projects in Unity 2020 and later.)
 - a. Double click on **Assets\Plugins\AnimationUprising\Strider\Integrations\StriderMxMIntegration** and click import to import StriderBipedMxM integration.
 - b. Double click on **Assets\Plugins\Threepeat\MMCLocomotion\Integrations\Strider_Integration** and click import to import MMCLocomotion/Strider integration.
11. (Optional, but highly recommended) Install Final IK
 - a. At time of writing this documentation, importing Final IK transient warnings/errors, which can be cleared and ignored as they are self-correcting.
 - b. Double click on **Assets\Plugins\Threepeat\MMCLocomotion\Integrations\FinalIK_Integration** and click import to import MMCLocomotion/Strider integration.

2.5 Installation steps - 2 - Project Settings

1. Make sure your ground layers are set up (Demos expect Layer 10 to be GrounderOnly and 11 to be GrounderIgnore, but that can be adjusted as needed by changing the layers of all the stairs and prism objects in the demo environment prefab to whichever layers your project uses for GrounderOnly and GrounderIgnore, respectively). See [Configuration - Final IK Grounder](#) for more information.

2.6 Installation steps - 3

1. Install MMLC from the Package Manager.
 - a. Import MMLP UnityPackage, confirm no errors in console.
2. If you see pink materials in any of the prefabs or demo scenes: Upgrade all materials to HDRP: Edit -> Rendering -> Materials -> Convert all Built-in Materials to (HDRP/URP)

3. Open the relevant example scene for your configuration to confirm all advertised functions work as expected before moving on to setting up custom characters and behavior. The table below shows appropriate example scene based on installed dependencies (MxM and MAP are required and assumed to be installed):

Strider	Final IK	Demo Scene (in Assets/Plugins/Threepeat/MMCLocomotion)
-	-	Demos/MMLCDemo_Core
✓	-	Demos/MMLCDemo_Core_Strider
-	✓	Demos/MMLCDemo_Core_FinalIK
✓	✓	Demos/MMLCDemo_Core_Strider_FinalIK

Information on key bindings and features of the demo scenes can be found here: [Demo Scene](#)

A video walkthrough ofTesting the Installation can be found [here](#):



2.7 Installation - Optional Animation Sets

Each Animation Set has 3 steps. Kubold's Movement Animset Pro (MAP) will be used for this purpose, but Female Movement Animset Pro (FMAP), Mocap Online's Mobility Pro (MOB) can be used interchangeably:

Prior to installation of the animation set asset, the desired animation set should appear as a single line in Config Wizard's Dependency Checks Tab (See `Optional Animation Set Integrations -> Movement Animset Pro (MAP) (Optional)...`):

InspectorLightingMMC Configuration Wizard

1. Dependency Checks

2. Setup Character

Help and Utilities

Core MMLC

✓ Motion Matching for Unity: Found → GOOD

Optional Animation Set Integrations

✗ Female Movement Animset Pro (FMAP) (Optional): Status: Main Pkg not installed, Integration Pkg not installed

Toggle Help

Open Package Manager

Ping Integration Package

✗ Movement Animset Pro (MAP) (Optional): Status: Main Pkg not installed, Integration Pkg not installed

Toggle Help

Open Package Manager

Ping Integration Package

✗ Mobility Pro (MOB) (Optional): Status: Main Pkg not installed, Integration Pkg not installed

Toggle Help

Open Package Manager

Ping Integration Package

iStep (Foot Placement System)

✗ iStep - Asset (Optional): Status: Main Pkg not installed

Toggle Help

Open Package Manager

✗ iStep - MMLC Integration Package (Optional): Status: Integration Pkg not installed

Toggle Help

Ping Integration Package

Strider

✗ Strider 1 of 2 (Optional): Status: Main Pkg not installed, Integration Pkg not installed

Toggle Help

Open Package Manager

Ping Integration Package

✗ Strider 2 of 2 (MMLC Integration) (Optional): Strider/MMLC Integration Pkg not installed

Toggle Help

Ping Integration Package

Final IK

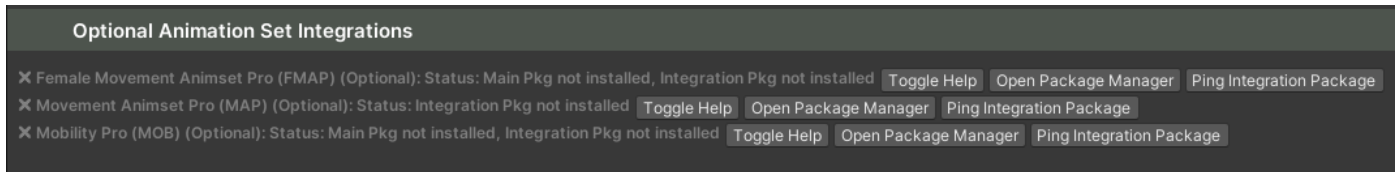
✗ Final IK (Optional): Not found or integration package not installed.

Toggle Help

Ping Integration Package

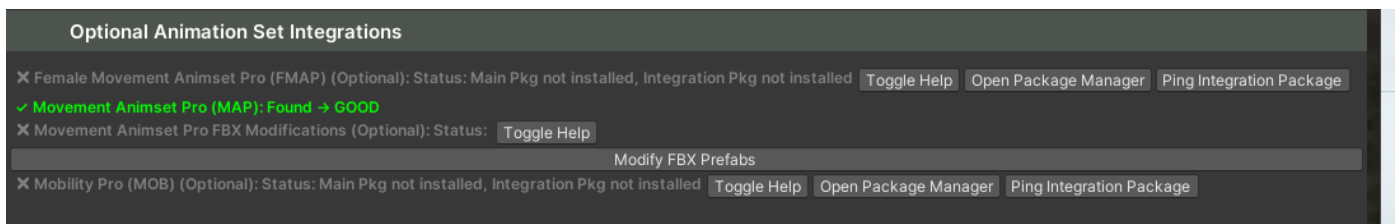
1. Import Animation Set Asset: e.g. Movement Animset Pro from Kubold (can delete all scripts and playmaker stuff, since that's not needed and just causes errors)
 - a. To avoid Playmaker-related errors either uncheck **MovementAnimsetPro/PlayMakerAdditionalActions** from the **Import Unity Package** dialog, or delete that directory after import.

The Movement Animset Pro line will remain grayed out, but the reason will change from `Main Pkg not installed, Integration Pkg not installed` to just `Integration Pkg not installed`:



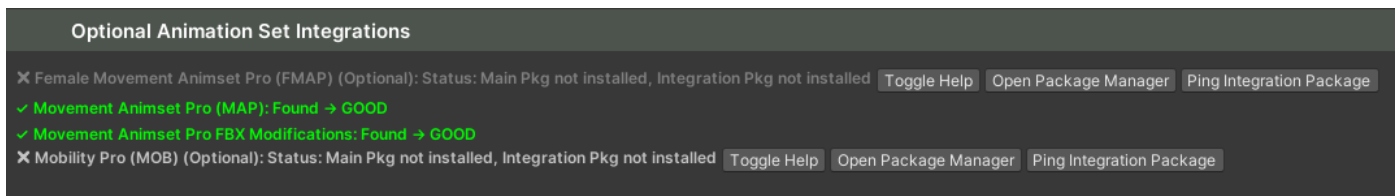
2. Click `Ping Integration Package` and double click the `MovementAnimsetPro_Integration` unity package to install the MMLC integration package.

The Movement Animset Pro line will turn green, showing the dependency is fully added and ready to use. There's still one more step to modify the FBX files to fix animation settings for use with motion matching and to inject `NGFootstepEvent` events for all of the footstep occurrences in the animation set.

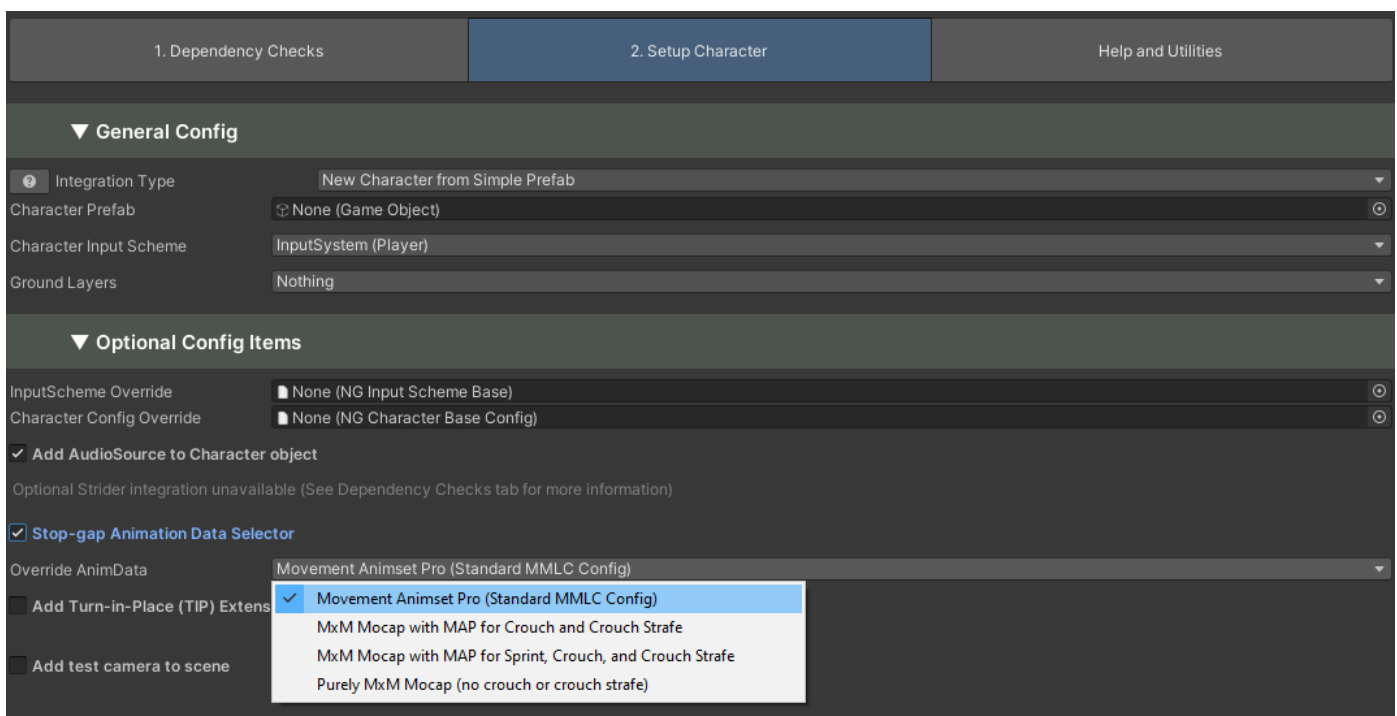


3. Click on `Modify FBX Prefabs` and confirm to modify the Movement Animset Pro prefabs for use with MMLC.

Now Both lines are green and you're fully ready to use MAP with MMLC.



4. In the Config Wizard -> Character Setup tab, you'll now see Movement Animset Pro available in the Stopgap animation data selector:



Repeat for any other animation sets.

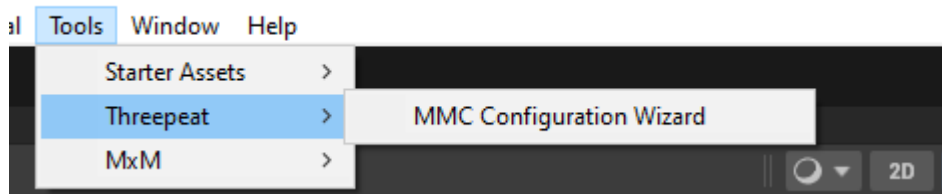
3. Configuration - Characters

3.1 Configuration Wizard

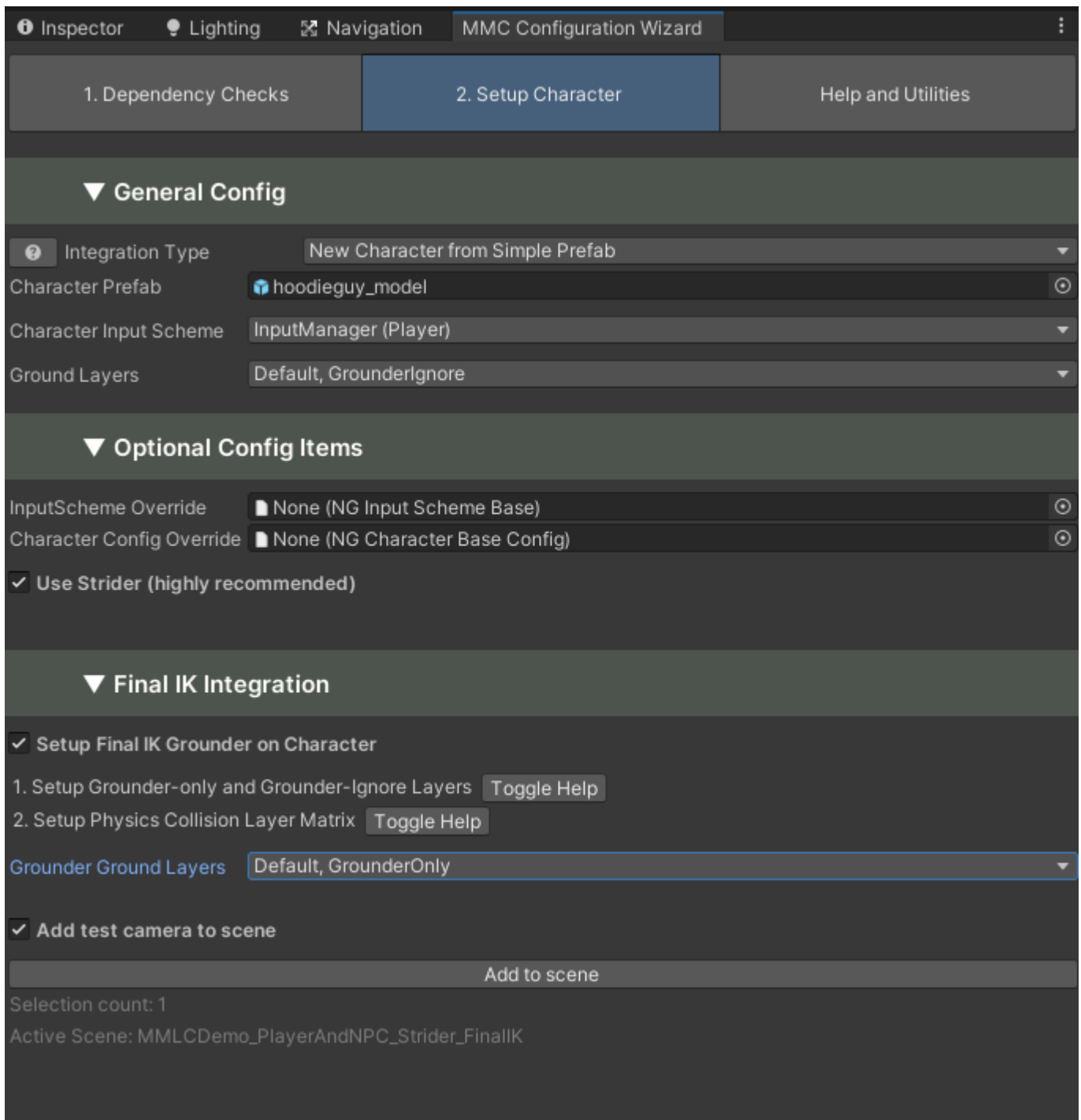
This asset comes with a configuration wizard that attempts to:

- Make player and Non-Player Character (NPC) setup automatic
- Automatically check and validate all dependency packages and integrations are installed in the project
- Assist with correcting any MMLC-related configuration issues in the project

To access the wizard, go to **Tools>Threepeat>MMC Configuration Wizard**



The wizard window should appear, as shown below (In image, the window has been docked with the Inspector):



The wizard has three main tabs: "Dependency Checks", "Setup Character", and "Help and Utilities"

Always start with Dependency Checks.

3.1.1 Dependency Checks

3.1.2 Help and Utilities

This tab contains a button to take you to the latest version of this help, which is located [here](#).

Additionally, this tab contains helper utilities. The only currently-exposed utility is "Add Footstep Events to Movement Animset Pro", which will insert all necessary **NGFootstepEvent** and **NGRawAudioEvent** Animation Events to make the necessary events in the **NGDispatchAnimationEvents** component fire.

This will allow you to add audio, visual effects, foot prints, and any other kind of reactions needed in response to foot steps. See [Audio Events](#) for more information

3.2 Setting up an input-driven character (Player)

The new InputSystem is highly recommended, but legacy InputManager is also supported by MMLC and the configuration wizard.

3.2.1 Joystick Support (InputManager)

1. The joystick left trigger is used for sprint, and should be named JoystickLeftTrigger and configured in **Project Settings -> Input Manager** as shown in the below picture:

The screenshot shows the Unity Input Manager configuration window. It is divided into two sections: JoystickLeftTrigger and JoystickRightTrigger. Each section has a list of properties on the left and corresponding input fields or dropdowns on the right.

Property	JoystickLeftTrigger Value	JoystickRightTrigger Value
Name	JoystickLeftTrigger	JoystickRightTrigger
Descriptive Name		
Descriptive Negative Name		
Negative Button		
Positive Button		
Alt Negative Button		
Alt Positive Button		
Gravity	0	0
Dead	0.1	0.1
Sensitivity	1	1
Snap	☐	☐
Invert	☐	☐
Type	Joystick Axis	Joystick Axis
Axis	9th axis (Joysticks)	10th axis (Joysticks)
Joy Num	Get Motion from all Joysticks	Get Motion from all Joysticks

1. In the PlayerCharacter's root object (the object with NGPlayerController component), set **use Joystick Axes** to true.

3.3 Setting up a Non-Player character (AI/NPC)

In addition to standard `NavMeshAgent::SetDestination` usage, `NGNonPlayerCharacter` offers a more persistent target transform that allows MMLC to manage reaction to target changes. This can be set/cleared as shown in **Assets/Plugins/Threepeat/MMCLocomotion/Scripts/Examples/Example_SetNavMeshTargetOnLKey**.

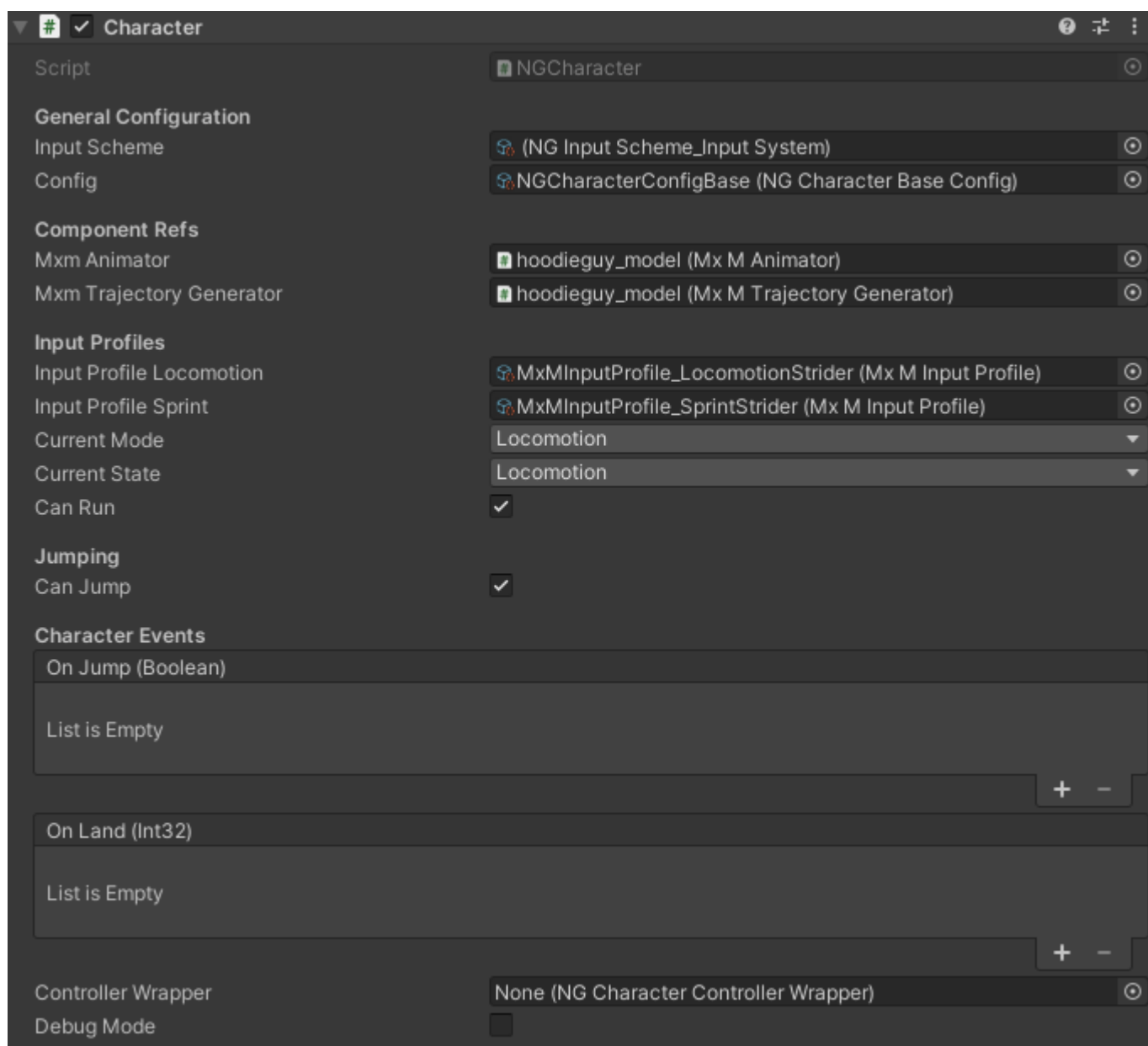
To change/clear a target for any NavMesh-driven character (code assumes it's running in a `MonoBehaviour` on the same `GameObject` as the `NGCharacter` component):

```
character = GetComponent<NGCharacter>();
NGInputScheme_NavMesh inputScheme = (NGInputScheme_NavMesh)character.InputScheme;
inputScheme.currentTarget = yourNewTargetTransform; // or null if you wish to clear the target.
```

3.4 Tuning Character Behavior for Players and NPC's

3.4.1 Player Only - Controls/Input

The `NGCharacter` component (and inspector) has an `Input Scheme` property that points to a `Scriptable Object` (see "Input Scheme" in image below) that controls all aspects of what drives a character to perform input-related actions. There are schemes for `InputSystem`, `InputManager`, and `NavMesh` (NPC driven by a target on the navmesh).



The inspectors for each of the three InputScheme types are below:


NG Input Scheme_Input System (NG Input Scheme_Input System)
Open

Script

General Settings

Do Environment Aware Trajectories ☒

Env Aware Minimum Dist To Obstacle

Input Vector Change Interval

InputSystem settings

Input Action Asset

Input Action Map Active

Action Name Movement

Action Name Jump

Action Name Sprint

Action Name Crouch

Jumping

Jump Max Key Hold Time Before Jump

Jump Instant Big Jump When Sprinting ☒


NG Input Scheme_Input Manager (NG Input Scheme_Input Manager)
Open

Script

General Settings

Do Environment Aware Trajectories ☒

Env Aware Minimum Dist To Obstacle

Input Vector Change Interval

InputManager settings

Move Axis Horizontal

Move Axis Vertical

Event Input Assignments

Keycode Crouch Primary

Keycode Crouch Secondary

Keycode Sprint

Keycode Jump Primary

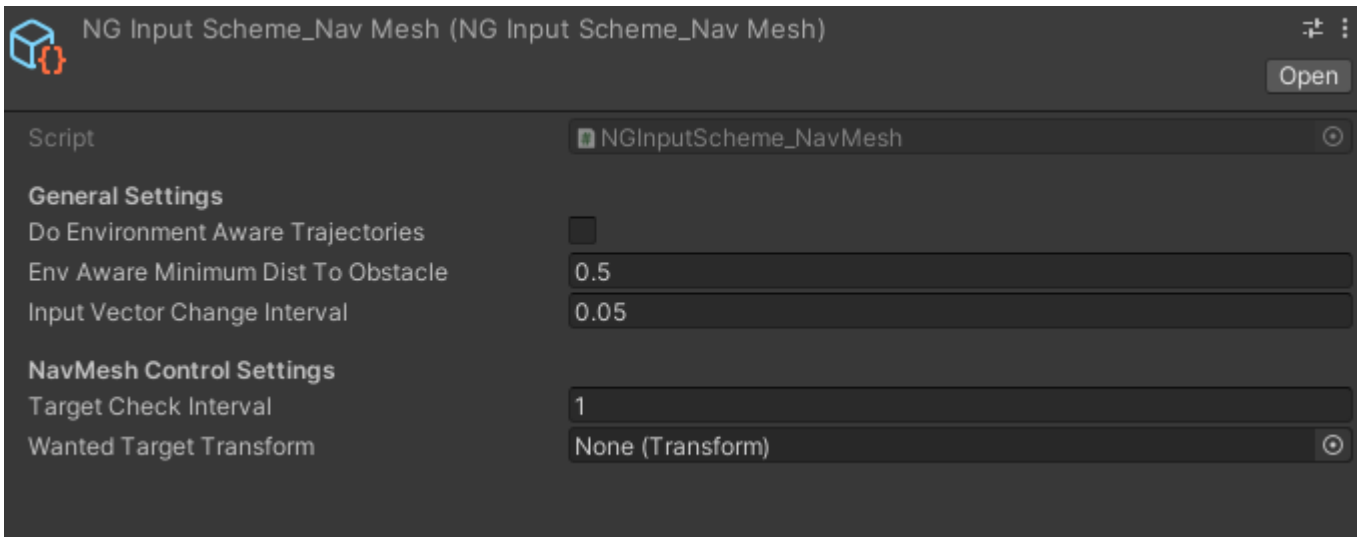
Keycode Jump Secondary

Use Joystick Axes ☐

Jumping

Jump Max Key Hold Time Before Jump

Jump Instant Big Jump When Sprinting ☒



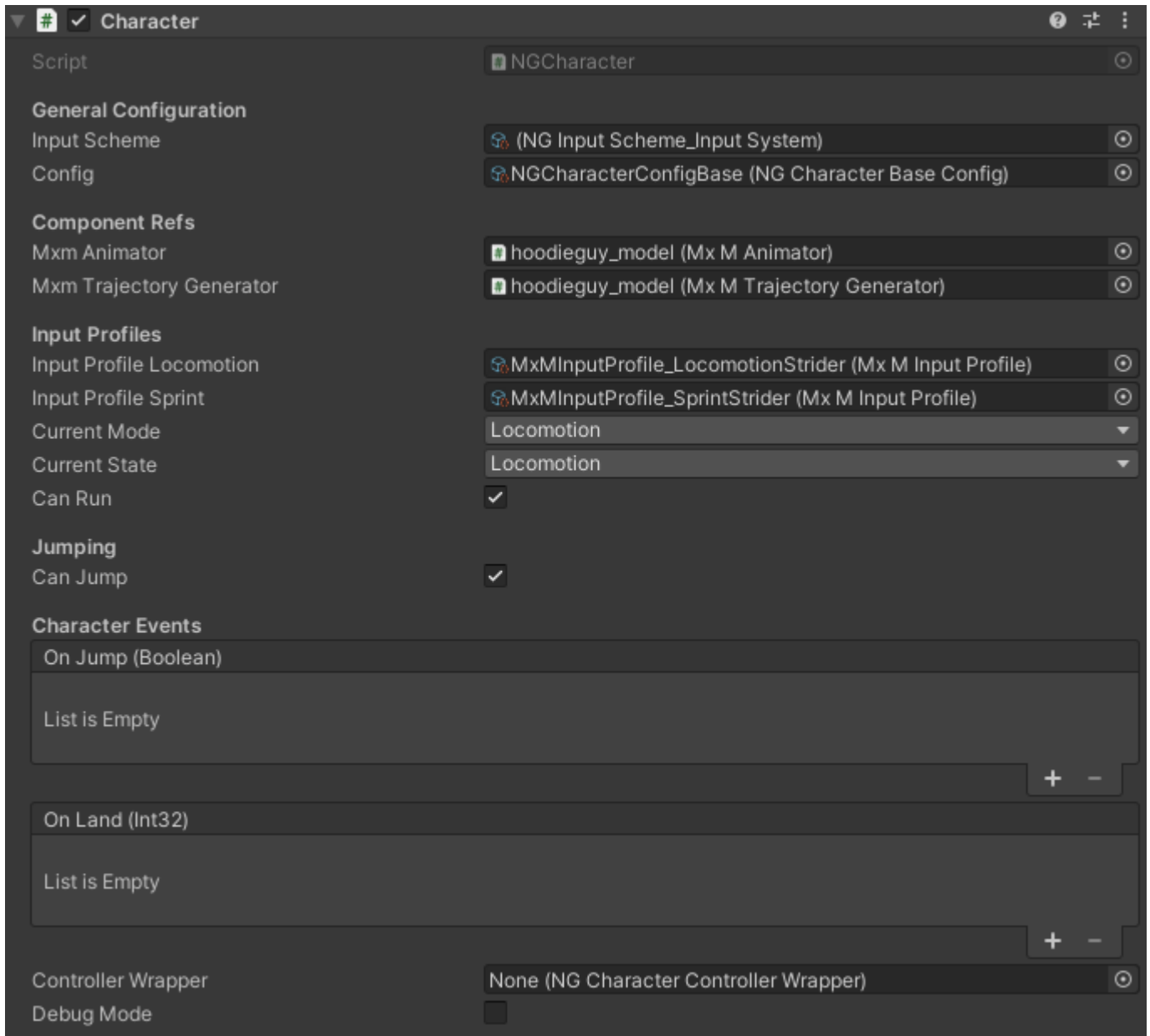
Important

Unlike the config property, a fresh input scheme object is instantiated (cloned) for each character, as otherwise, e.g. all common NavMesh-controlled agents would only be able to move to the same destination.

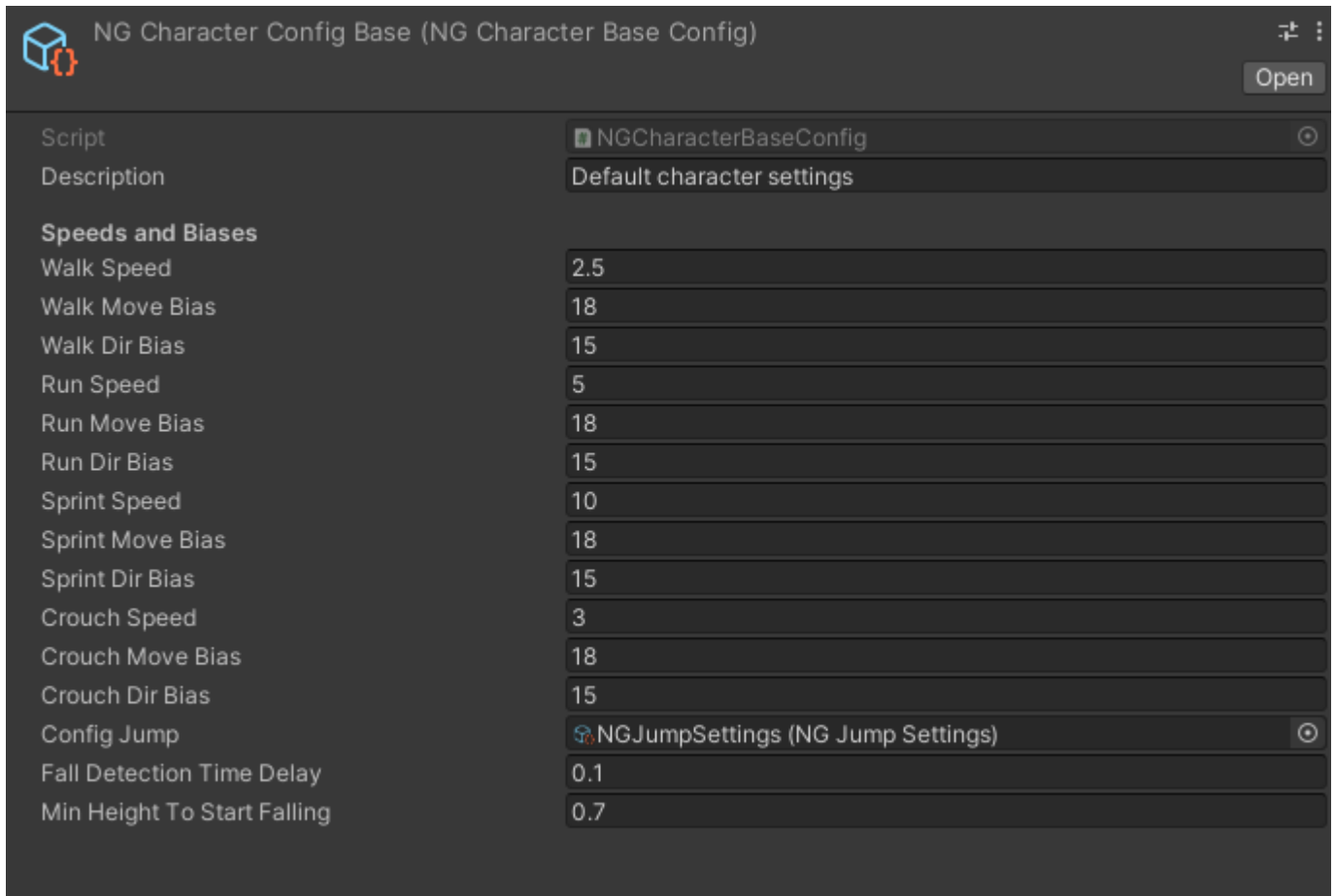
In the *Jumping** section of the InputSystem and InputManager inspectors, shown above, how long the jump key must be held down before a big jump is executed can be configured (set to 0 or a very small number of seconds (e.g. 0.001) for instant big jump), as well as whether to perform an instant big jump when the jump key is pressed while sprinting. In all other cases, the jump will occur on key up or after the max key hold time before jump has elapsed since the key was pressed.

3.4.2 Common - Run/Sprint/Crouch-walk speed, Jump enable/disable, Falling configuration

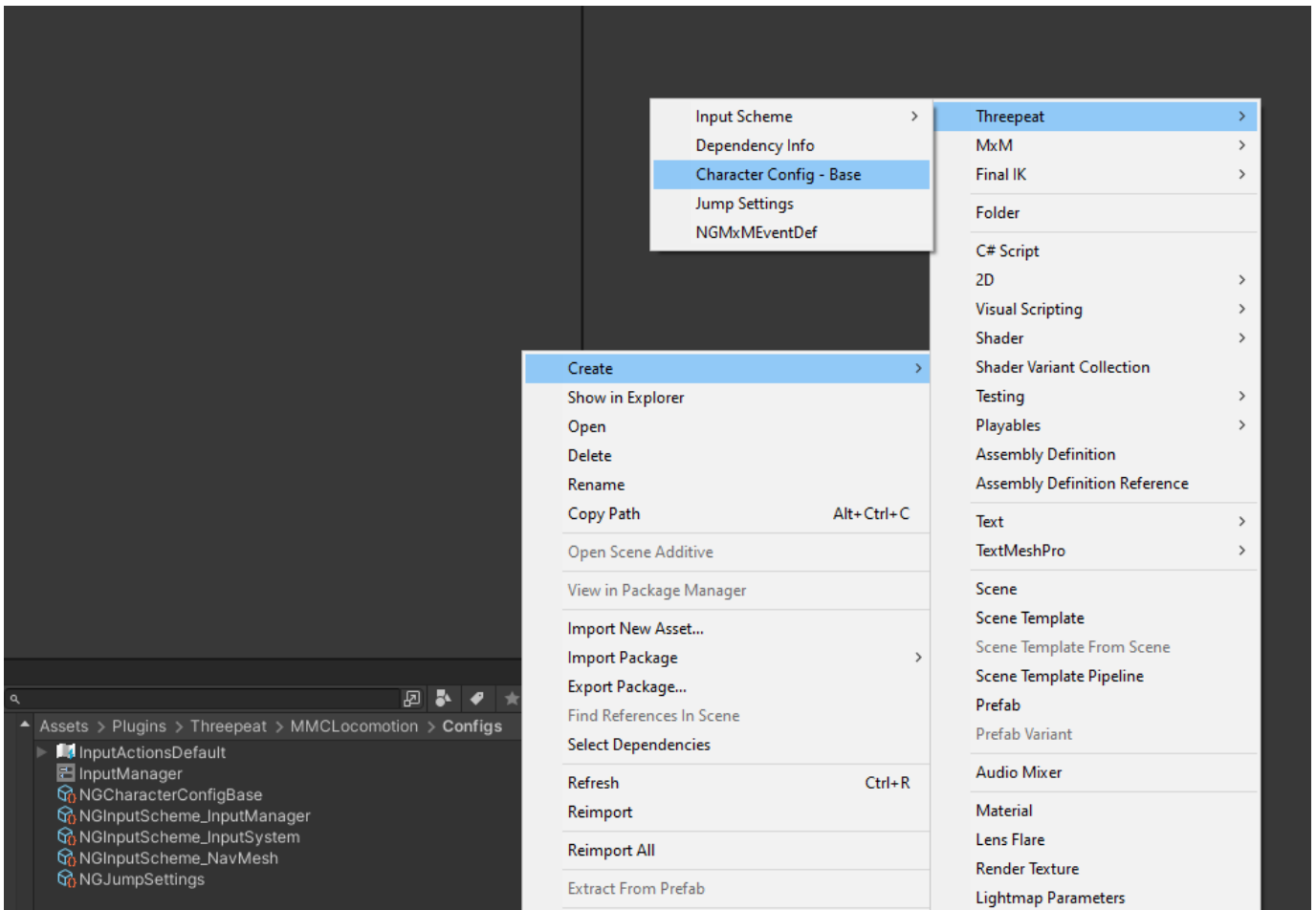
The NGCharacter component (and inspector) has a config property (see "Config" in image below) that points to a Scriptable Object that controls the character's locomotion and jumping behavior.



Here is the inspector for that Scriptable Object:



These can be found in **Assets/Plugins/Threepeat/MMCLocomotion/Configs** and created via right-click menu as shown below:



To adjust walk and run speeds of a character, change **Walk and Run Speed** in the **Speeds and Biases** section of the NGCharacter config scriptable object. For sprint, adjust **Sprint Speed**, and crouch-walk speed can be adjusted via **Crouch Speed**.

While jumping is predominantly configured via the reference NGJumpSettings ScriptableObject (discussed in next section), the **Jumping** section of the controller contains the toggle for whether a character is able to jump at all.

For falling detection, **Fall Detection Time Delay** configured the time after the character is no longer grounded (there is no longer any ground surface directly under their feet) before the character goes into a "Falling" state (with accompanying fall- and forward- speed-based animations). **Min Height To Start Falling** defines the minimum distance between the character's feet and the ground for falling to be engaged.

Note

Since v0.2.0 on, All common settings (applicable to both Player and NPC characters) have been rolled into a top-level NGCharacterConfigBase ScriptableObject to make character classes/types more portable, easier to define, and easier to drop into characters at scale. This also allows wholesale changing of behavior for all commonly-configured characters with a single object modification.

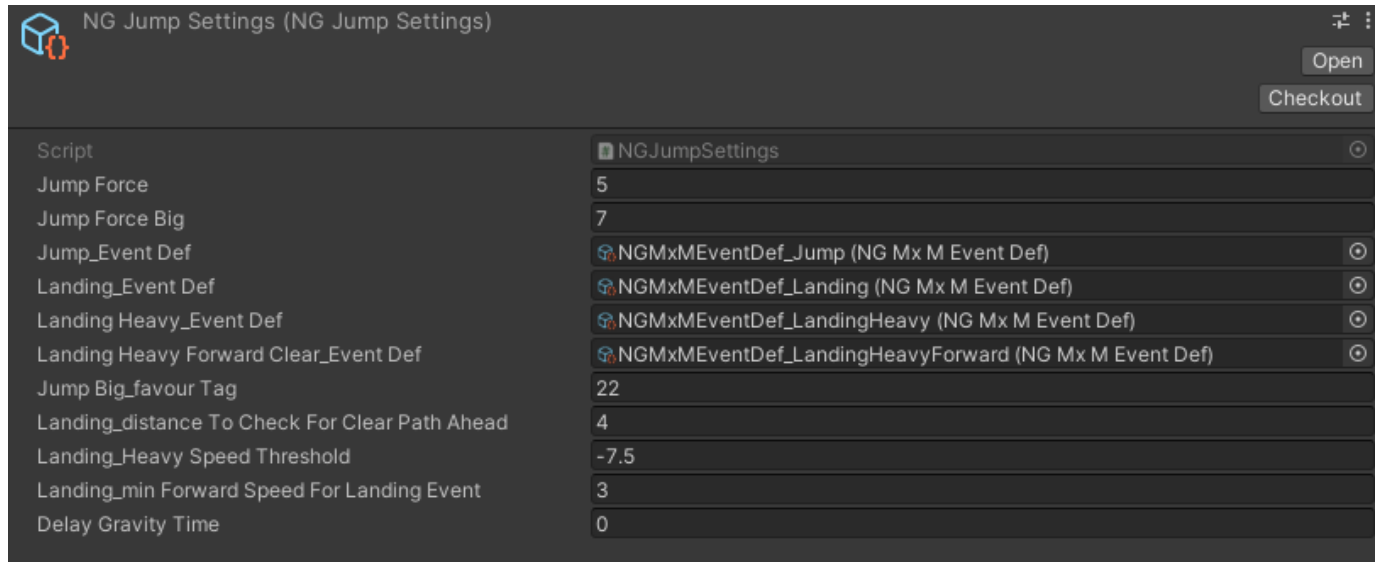
The inspector screenshot above also highlights the two major events available in the core character class:

- On Jump (boolean: true if big jump, false otherwise)
- On Land (int: int value of NGCharacterBase.LandingType enum which defines the types of landing available in the controller)

3.4.3 Common - Deeper Jumping Configuration

The below NGJumpSettings ScriptableObject allows different jump behavior to be configured. The items can be configured:

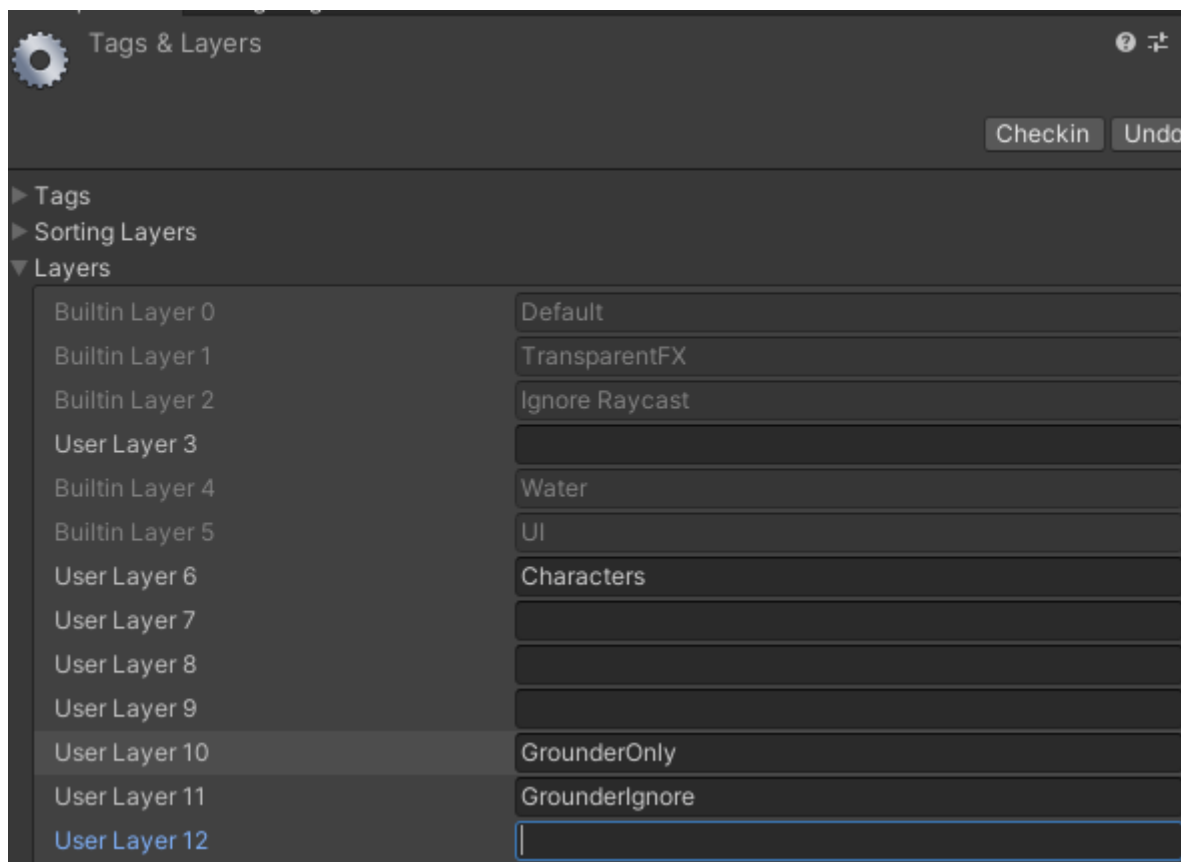
- **Jump Force:** upward force to apply for regular jumps
- **Jump Force Big:** upward force to apply for big jumps
- **Landing_Distance to Check for Clear Path Ahead:** Distance ahead of character at landing to check for obstructions. If no obstructions found, the player will perform a landing roll. Otherwise, the player will perform a stationary heavy landing.
- **Landing_Heavy Speed Threshold:** Minimum downward speed (ground impact speed) to trigger a "Heavy" landing.
- **min Forward Speed For Landing Event:** For normal landings below the heavy speed threshold, this defines the minimum forward speed to engage moving landing animations.



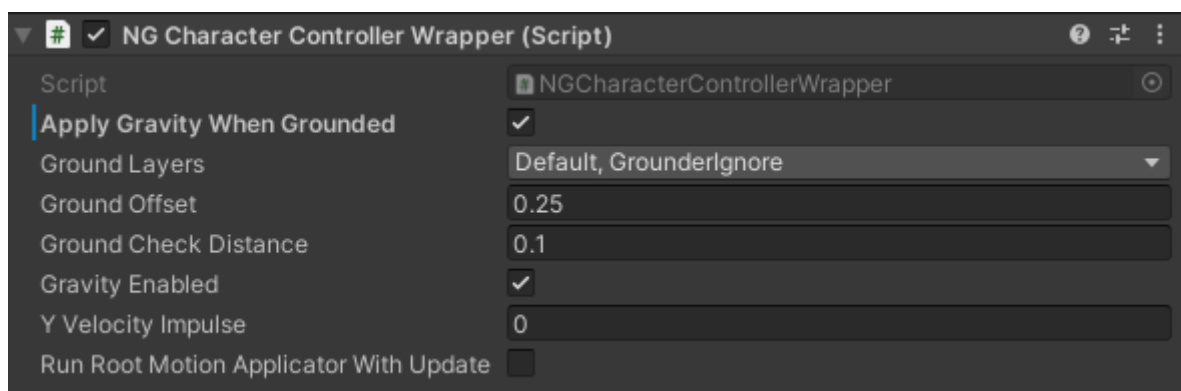
3.5 Setting up your Character and Environment for Final IK grounder

3.5.1 Environment

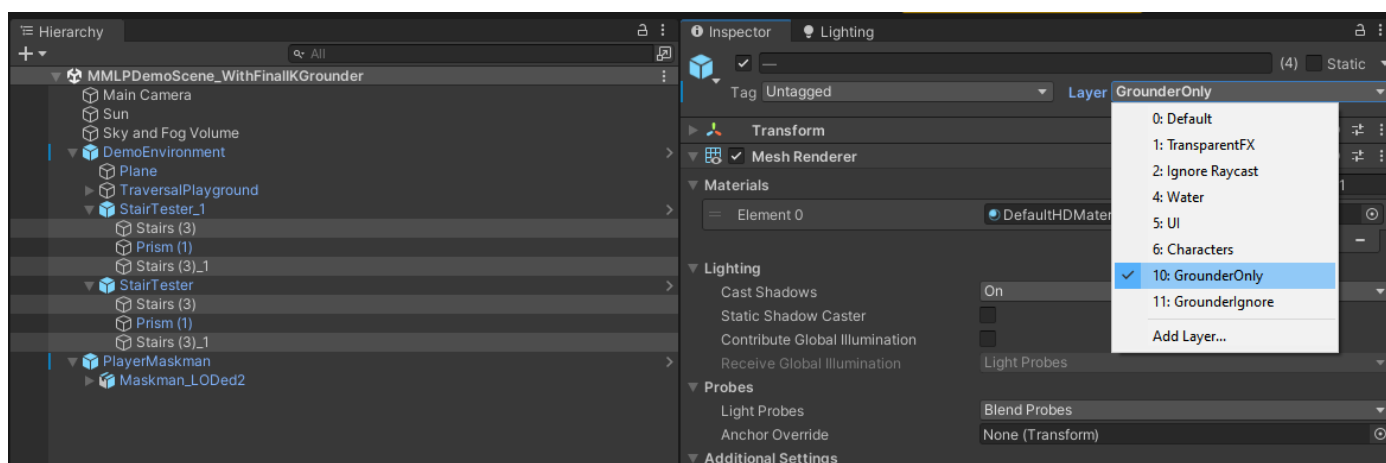
1. Create Layers: GrounderOnly and GrounderIgnore



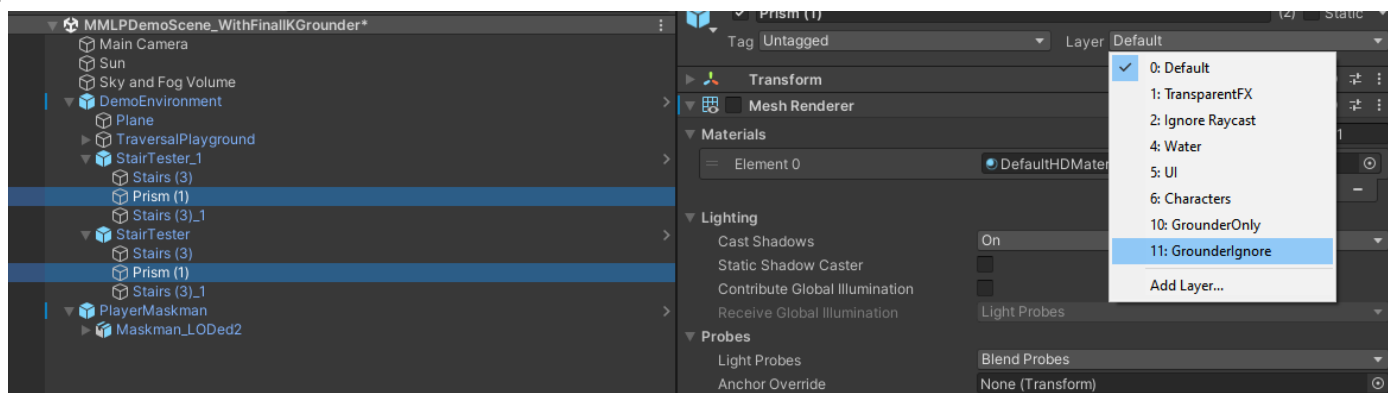
2. Add GrounderIgnore layer to Ground Layers in NGCharacterControllerWrapper in your Top-level player object (e.g. **hoodieguy_model**)



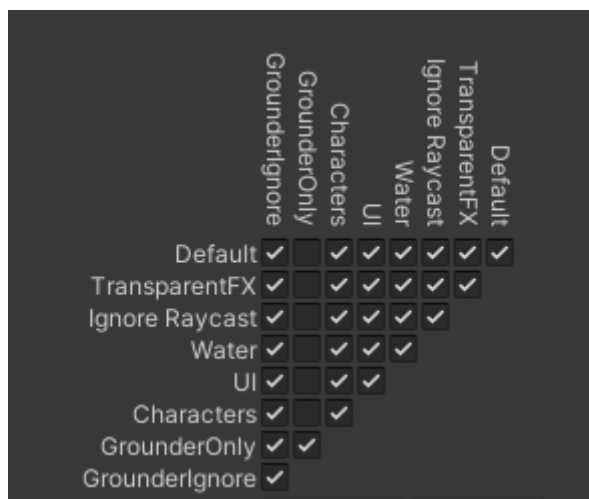
3. Put all stepped-inline objects (stairs) on **Grounder Only** layer



4. Put the smooth inclines that should actually drive non-jittery locomotion on **Grounder Ignore** layer



5. In Project Settings -> Physics, disable all physics collisions between other layers and **Grounder Only**:

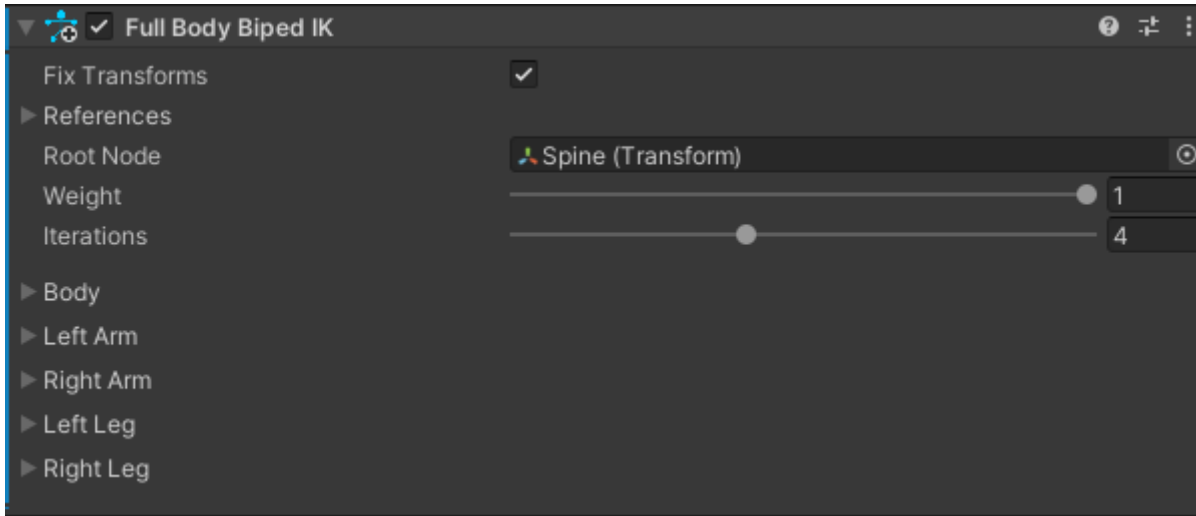


3.5.2 Character

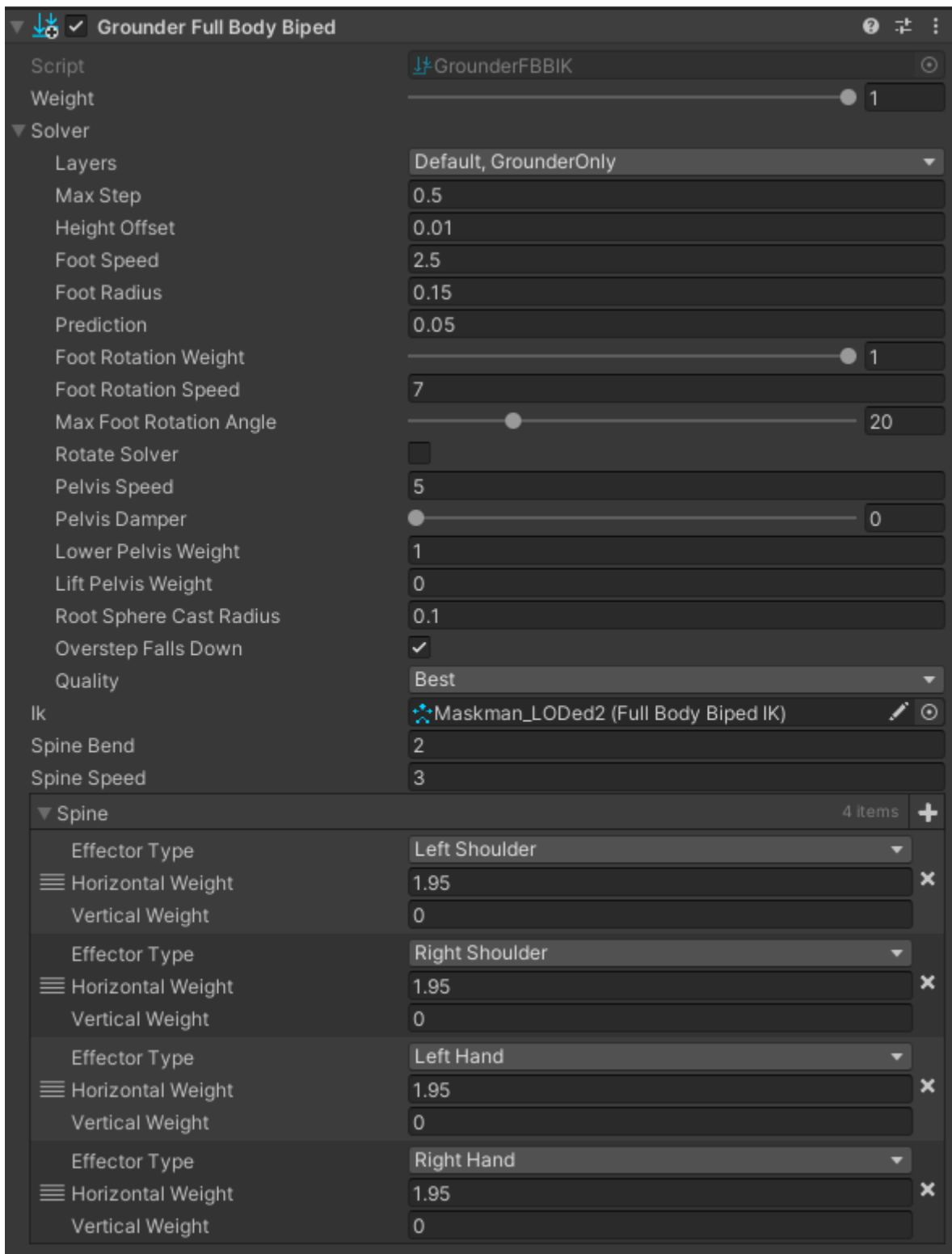
Important

This section is provided for general reference. If you're using our Motion-Matching Locomotion Controller, the config wizard will automatically configure The Biped IK and Grounder components for you. However, if you wish to perform this step manually, or if you don't any Threepeat assets and just want to set up grounder, here you go:

1. To the sub-object of your character that contains the Animator (and MxMAnimator, etc), Add **FBBIK** Component:



2. To that same object, add **Grounder Full Body Biped** component and set settings per the picture below:

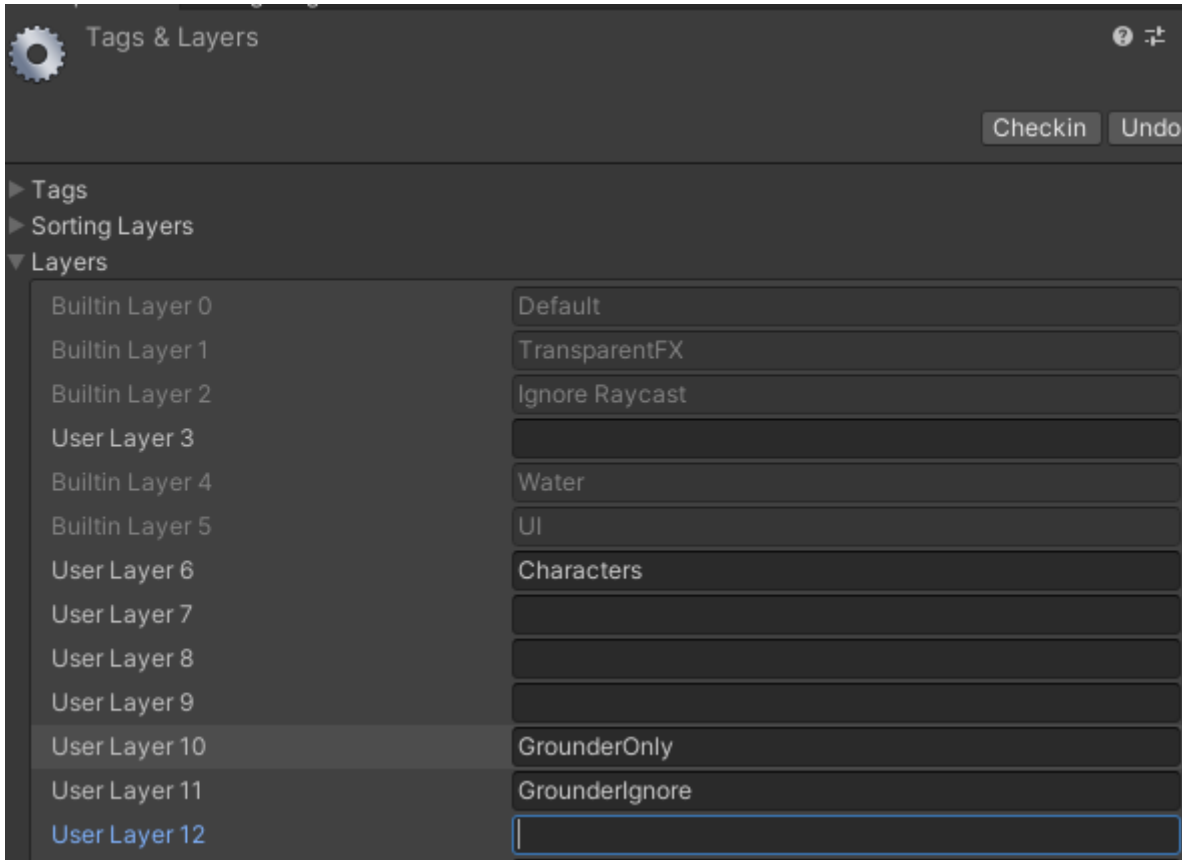


- Make sure All ground layers (except Grounder Ignore) are set in Solver -> Layers
 - Add the Left/Right Shoulders and Left/Right Hands to the Spine list (this causes the player to lean forward when running up inclines and stay more upright when descending inclines to add realism)
3. You're done!

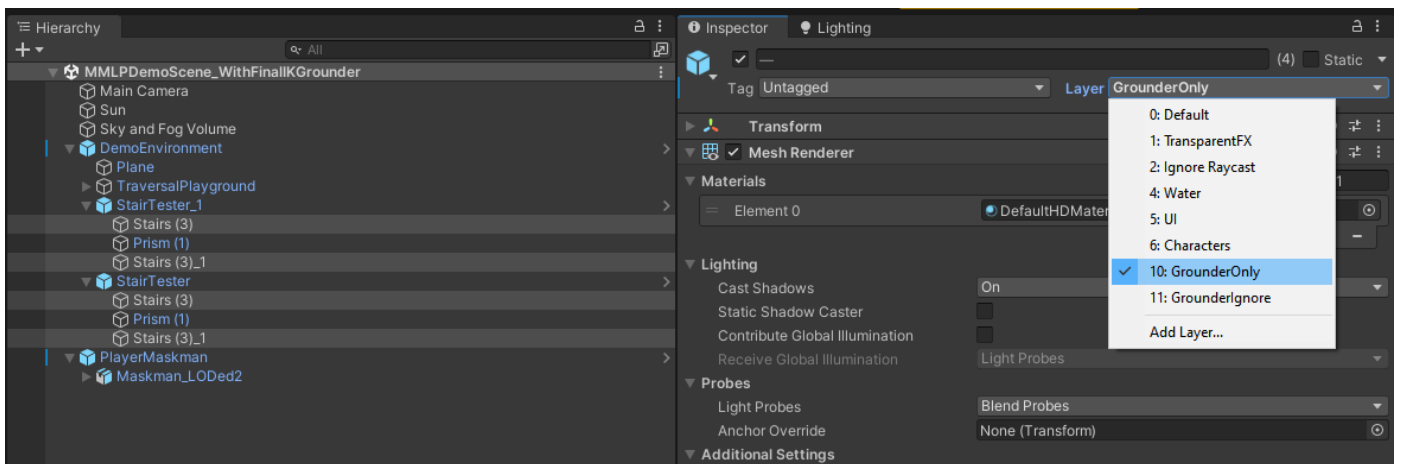
3.6 Using any of the demo scenes that include Final IK

A version of the Core Demo with Final IK grounder integration is in the Demos directory. This may not work directly after import (due to required Layers not being set up in the project), to resolve this:

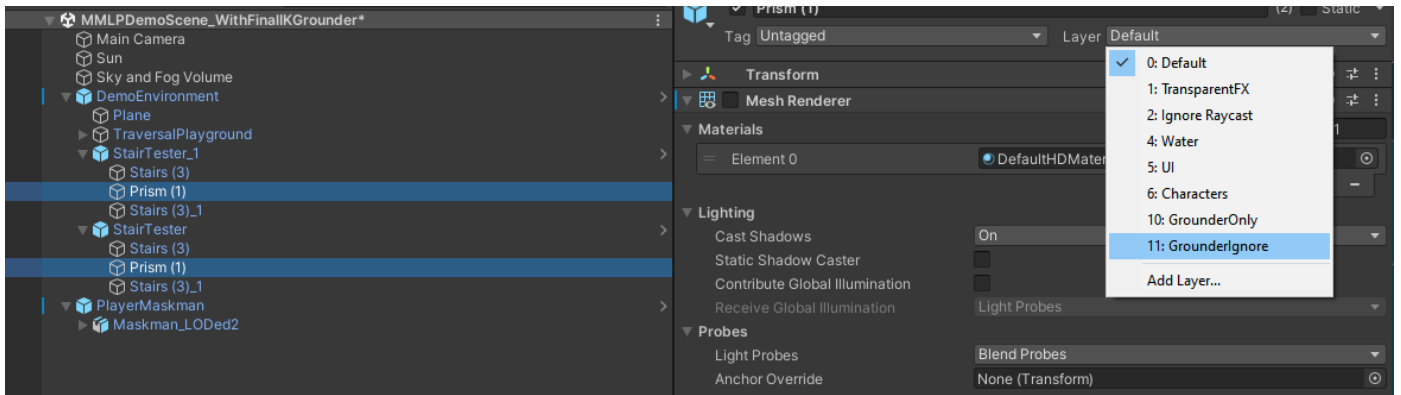
1. Create Layers: GrounderOnly and GrounderIgnore



2. Put all stepped-incline objects (stairs) on **Grounder Only** layer



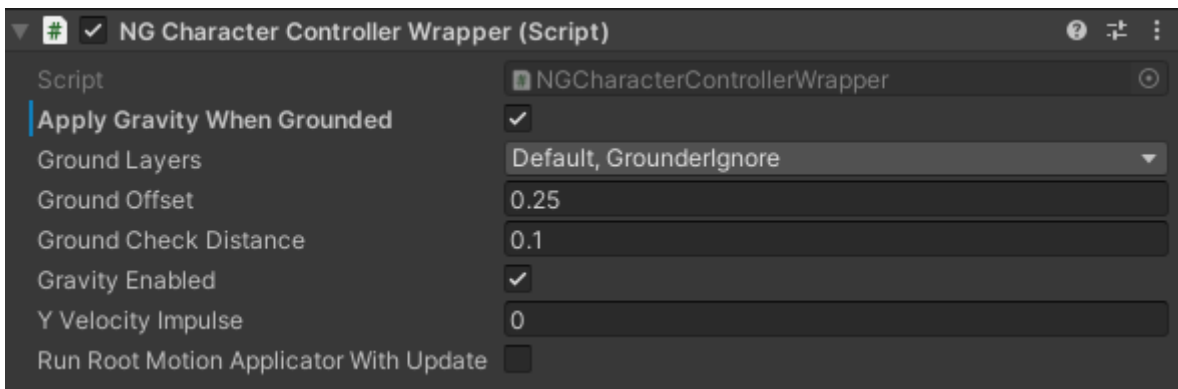
3. Put the smooth inclines that should actually drive non-jittery locomotion on **Grounder Ignore** layer



4. In Project Settings -> Physics, disable all physics collisions between other layers and **Grounder Only**:



5. Add GrounderIgnore layer to Ground Layers in NGCharacterControllerWrapper in your Top-level player object (e.g. **hoodieguy_model**)

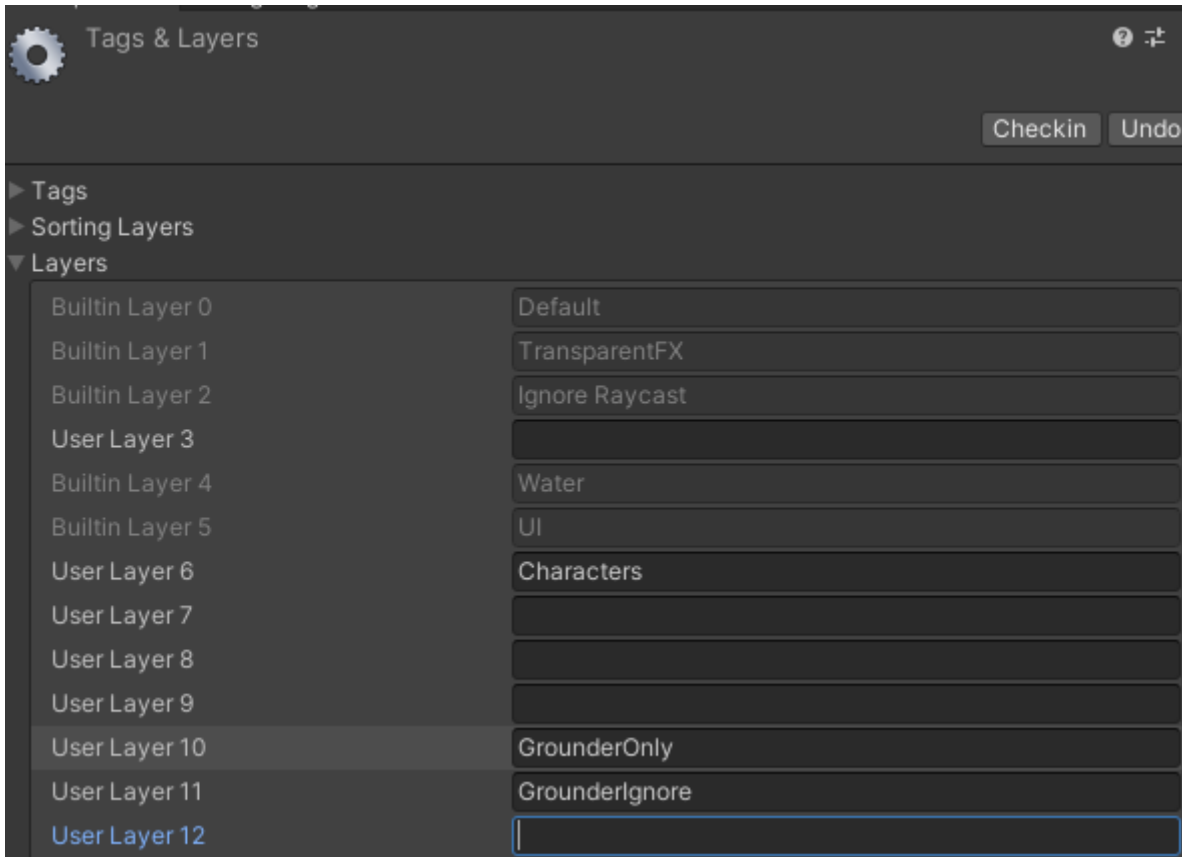


4. Configuration - Final IK Grounder

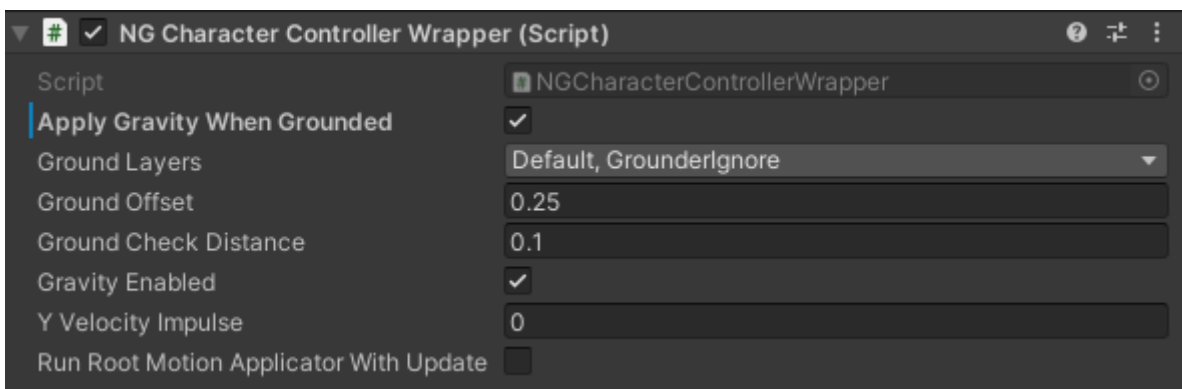
4.1 Setting up your Character and Environment for Final IK grounder

4.1.1 Environment

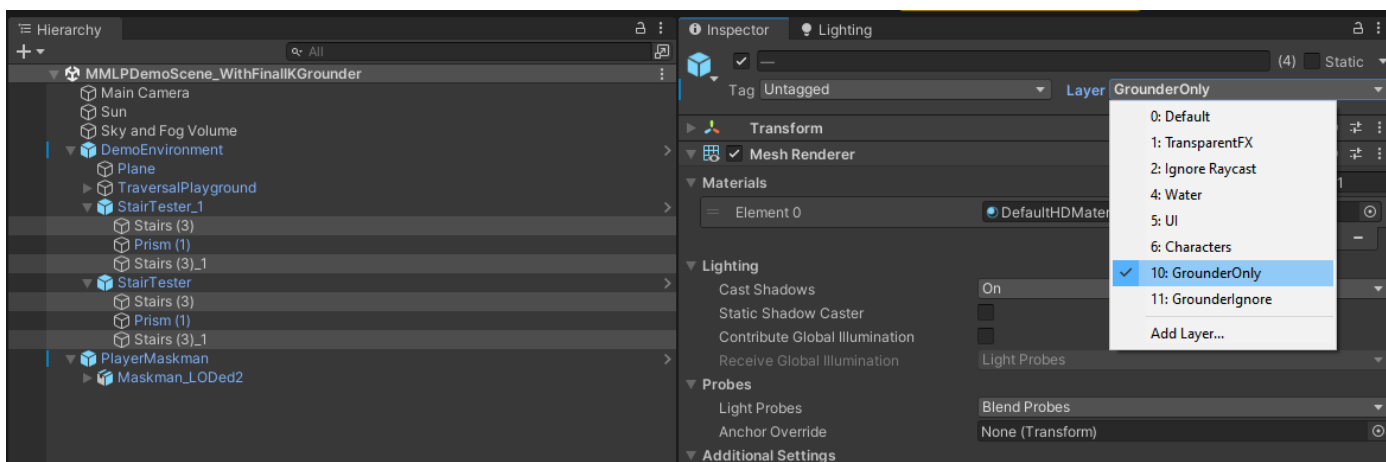
1. Create Layers: GrounderOnly and GrounderIgnore



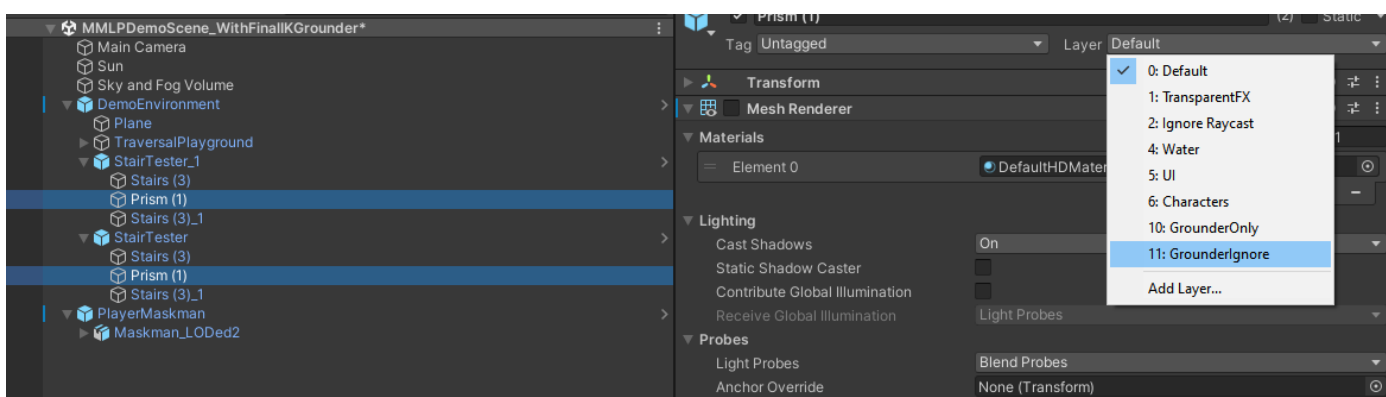
2. Add GrounderIgnore layer to Ground Layers in NGCharacterControllerWrapper in your Top-level player object (e.g. `hoodieguy_model`)



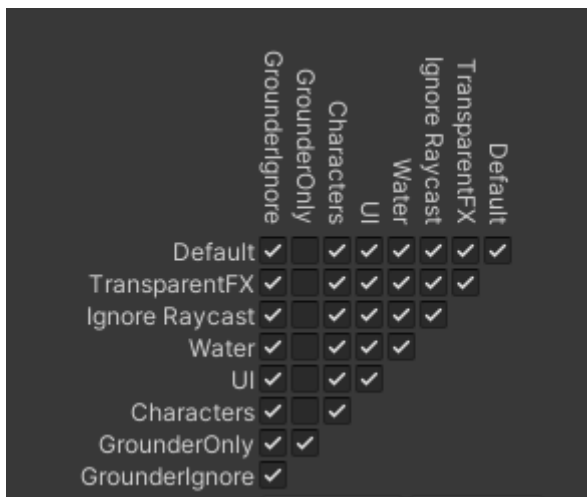
3. Put all stepped-inline objects (stairs) on **Grounder Only** layer



4. Put put the smooth inclines that should actually drive non-jittery locomotion on **Grounder Ignore** layer



5. In Project Settings -> Physics, disable all physics collisions between other layers and **Grounder Only**:

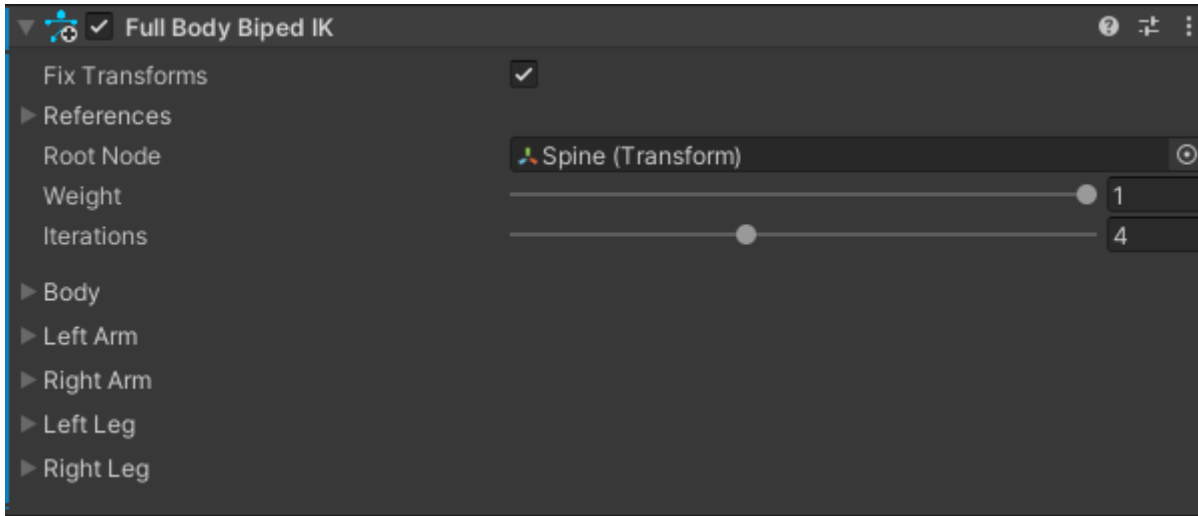


4.1.2 Character

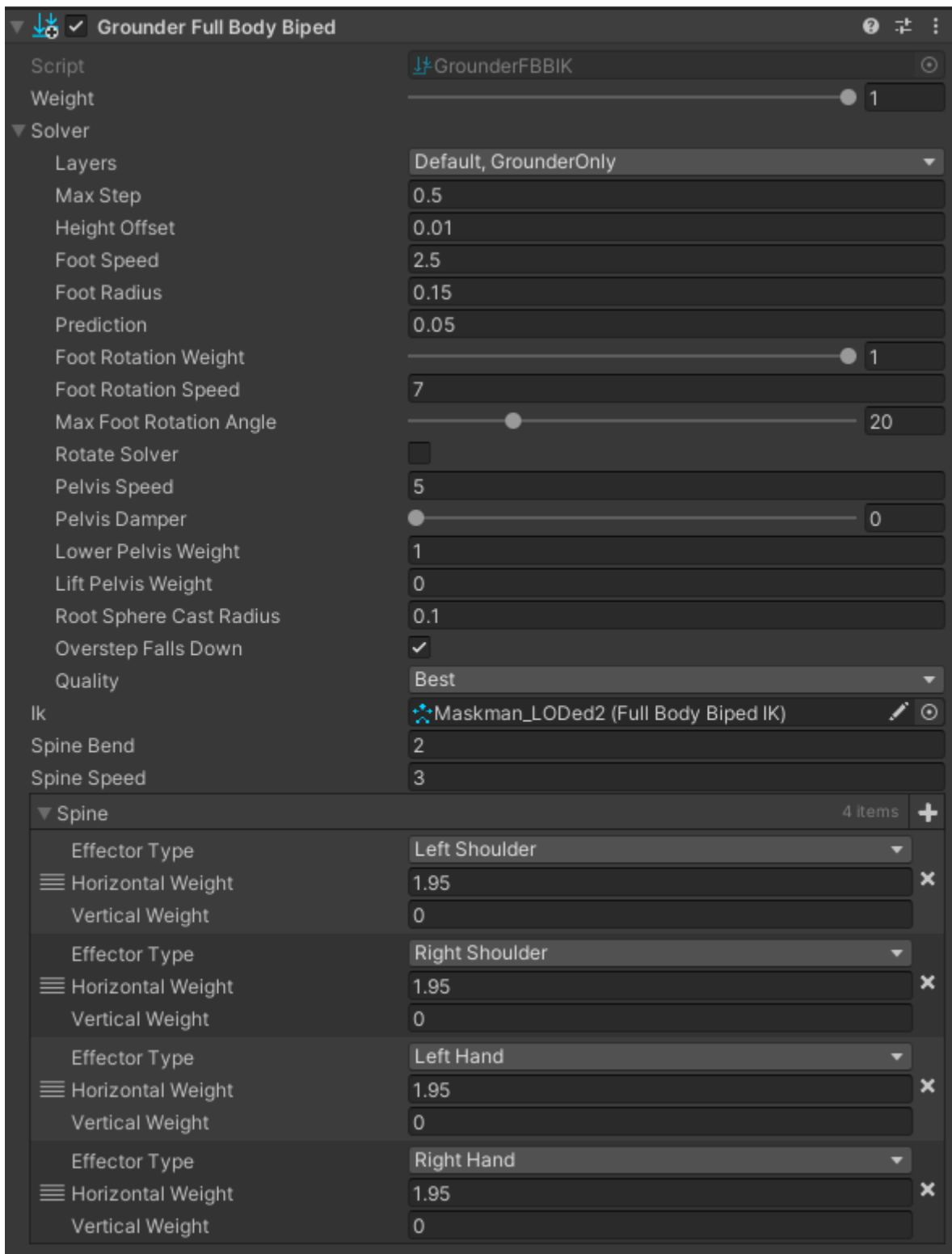
Important

This section is provided for general reference. If you're using our Motion-Matching Locomotion Controller, the config wizard will automatically configure The Biped IK and Grounder components for you. However, if you wish to perform this step manually, or if you don't any Threepeat assets and just want to set up grounder, here you go:

1. To the sub-object of your character that contains the Animator (and MxMAnimator, etc), Add **FBBIK** Component:



2. To that same object, add **Grounder Full Body Biped** component and set settings per the picture below:

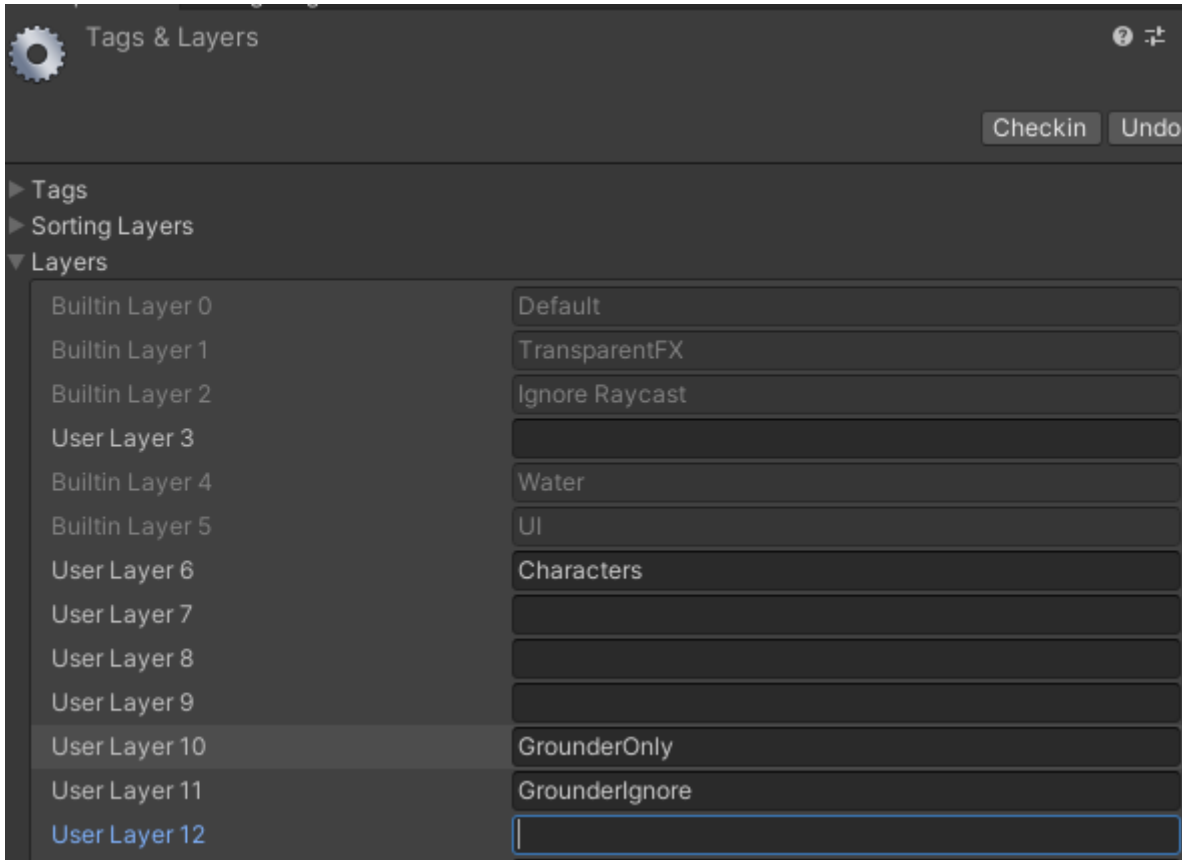


- Make sure All ground layers (except Grounder Ignore) are set in Solver -> Layers
 - Add the Left/Right Shoulders and Left/Right Hands to the Spine list (this causes the player to lean forward when running up inclines and stay more upright when descending inclines to add realism)
3. You're done!

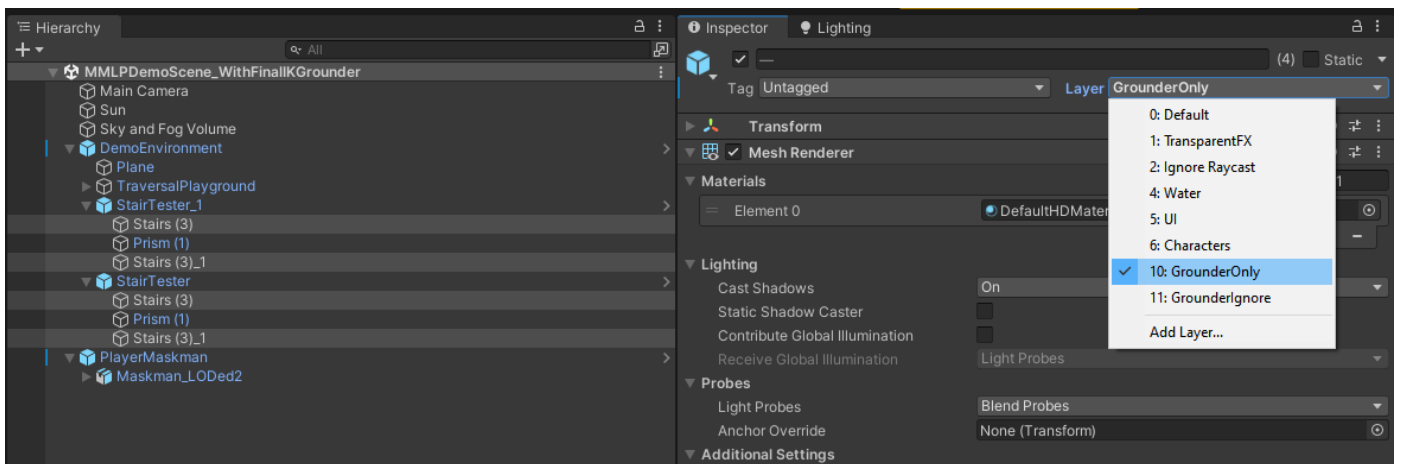
4.2 Using any of the demo scenes that include Final IK

A version of the Core Demo with Final IK grounder integration is in the Demos directory. This may not work directly after import (due to required Layers not being set up in the project), to resolve this:

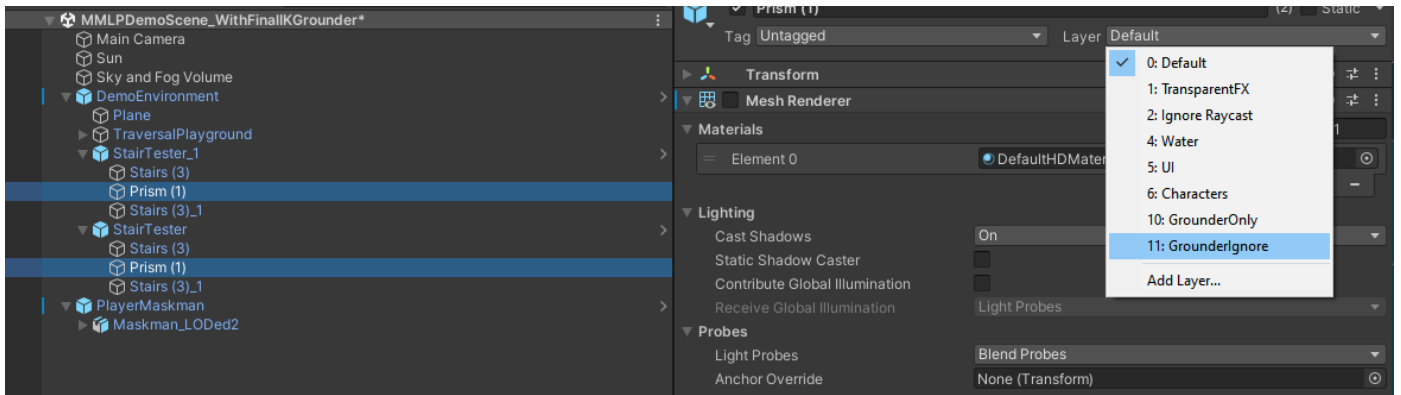
1. Create Layers: GrounderOnly and GrounderIgnore



2. Put all stepped-incline objects (stairs) on **Grounder Only** layer



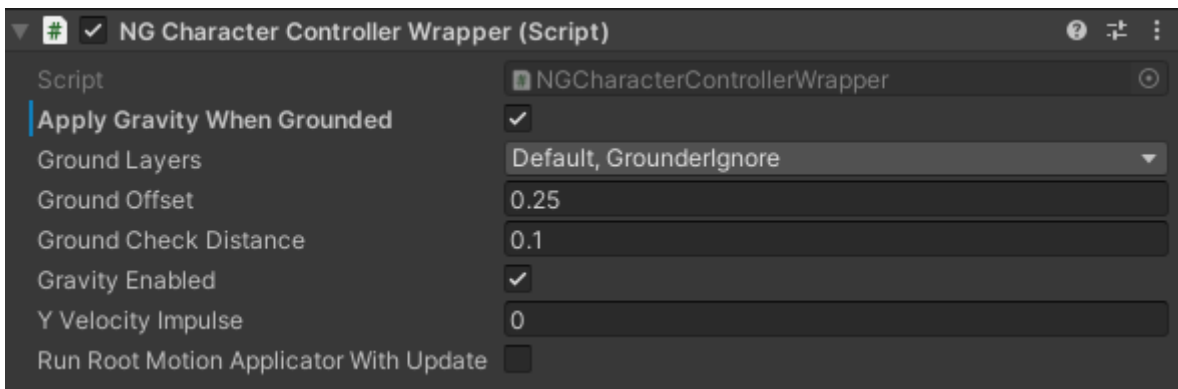
3. Put the smooth inclines that should actually drive non-jittery locomotion on **Grounder Ignore** layer



4. In Project Settings -> Physics, disable all physics collisions between other layers and **Grounder Only**:



5. Add GrounderIgnore layer to Ground Layers in NGCharacterControllerWrapper in your Top-level player object (e.g. **hoodieguy_model**)

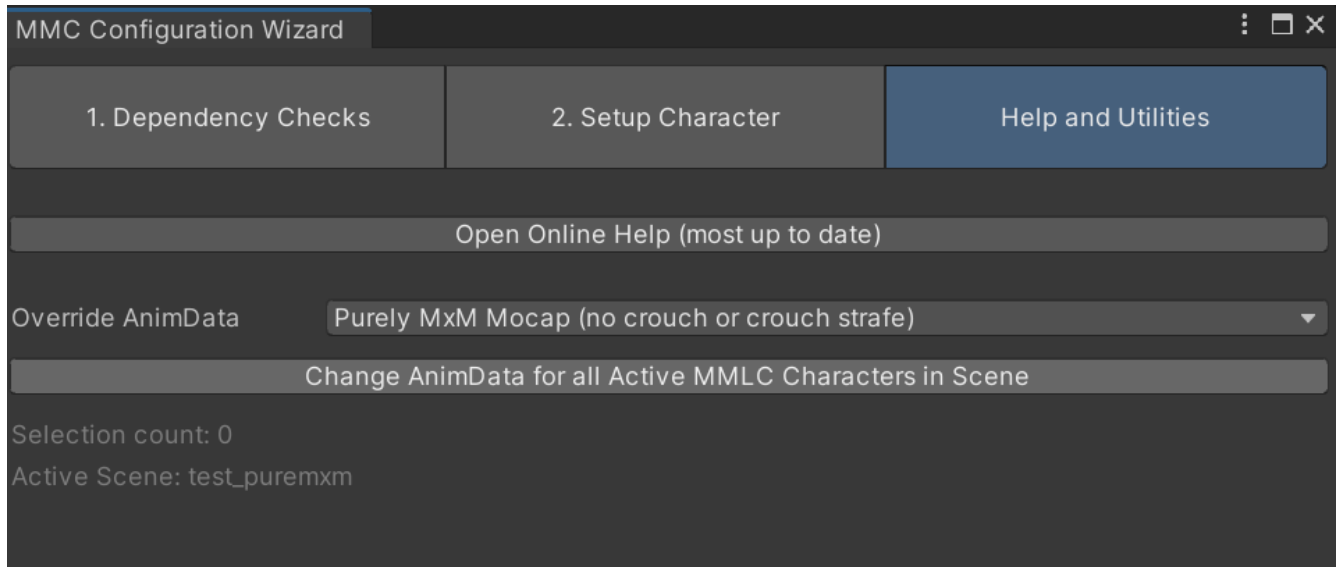


5. Demo Scenes and Examples

The MMLC unitypackage contains an example scenes (**Assets\Plugins\Threepeat\MMLocoParkour\Demos**) with a fully-configured Humanoid-rigged character to demonstrate all of the functions:

Update - Animation Set Change Helper tool Now Available!

There is now an option in `Config Wizard -> Help and Utilities` tab to change the animation set for all active/enabled MxmAnimator's (MMLC Characters) in the scene:



1. Select your desired animation set in the `Override AnimData` dropdown, then click `Change AnimData for all Active MMLC Characters in Scene.`
2. That's it!

 Important

The FinalIKGrounder demo scene will only work if you've setup your project and that scene as described in the **Using MMLPDemoScene_With_FinalIKGrounder Demo Scene** section of [Configuration - Final IK Grounder](#).

Behavior	Action (if needed)
Move camera	mouse
zoom camera	mouse wheel
Running	Arrow keys
smooth walk/run transitions	(Requires Strider for maximum smoothness) Use Gamepad, joystick, or any input axis with graduated [-1,1] values
Sprint	Left Shift (Hold)
Crouch	Left Ctrl - Only Available with Optional Animation Sets integrated
Crouch walk	Arrow keys or joystick while crouching - Only Available with Optional Animation Sets integrated
Jump	Spacebar
Big Jump	Hold Spacebar for > 0.35 seconds or press spacebar while sprinting
Fall detection	Run off of an elevated platform, player will detect falling and switch to Falling Blend-Space based on forward speed
Normal/Hard/ Rolling Landing	Based on impact speed with the ground when landing, Player will immediately resume whatever behavior they were doing prior to the jump or falling condition (Normal), execute a hard landing event with a forward roll (if the path ahead of them is clear), or execute a hard landing event in place (if path ahead of them is blocked)

 Note

If you arrived at this page from Installation Step 3, you can continue with configuring your own custom player character at [Configuration - Character](#)

5.1 Completed Examples

- Blending between MMLC and the animator (including with Avatar Mask to simultaneously use MMLC and the Animator): See **Example_ToggleAnimatorOnKey**
- Toggle whether character is able to run or only walk: See **Example_ForceWalkOnKey**
- Programmatic Jump triggering: See **Example_MakeCharacterJumpWhenYouPressGKey**
- Change a NavMesh-driven character's destination: See **Example_SetNavMeshTargetOnLKey**
- Switch between NavMesh and InputSystem or InputManager at runtime: See **Example_SwitchInputSchemeToInputSystemOnKey**, **Example_SwitchInputSchemeToNavMeshOnKey**, and **Example_SwitchInputSchemeToInputManagerOnKey**

5.2 Completed Tutorials

All tutorials are available on our Youtube: <https://www.youtube.com/@threepeatgames7837/videos>

5.3 Scripts from Youtube Tutorials

[Working with Mecanim Part 1](#), [Example_ToggleAnimatorTheNewWay.cs](#)